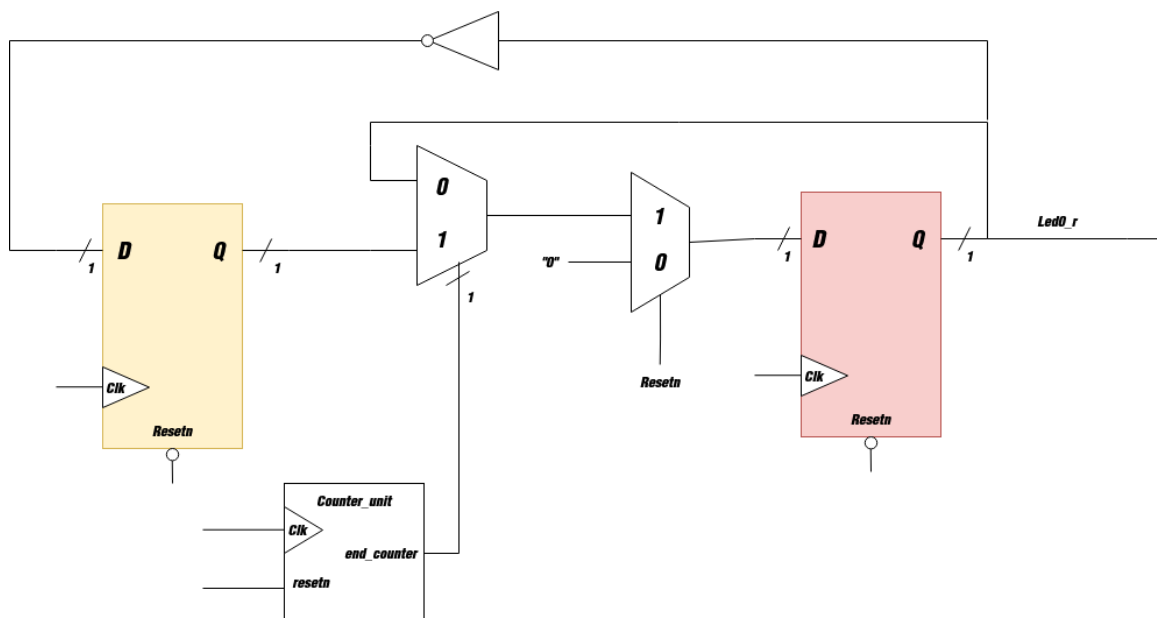


09/07/2026 TP4 : Pilotage led et mémoire.

Schéma RTL
Résultat de simulation avec chronogramme commentés
Vos résultats de synthèse (analyse de vos ressources)
Vos résultats de STA (analyse du rapport de timing)
Une démonstration de votre design

1. Créez une architecture RTL permettant de faire clignoter une LED (par exemple la led0_r) en utilisant le module Counter_unit du TP2 et une machine à état.



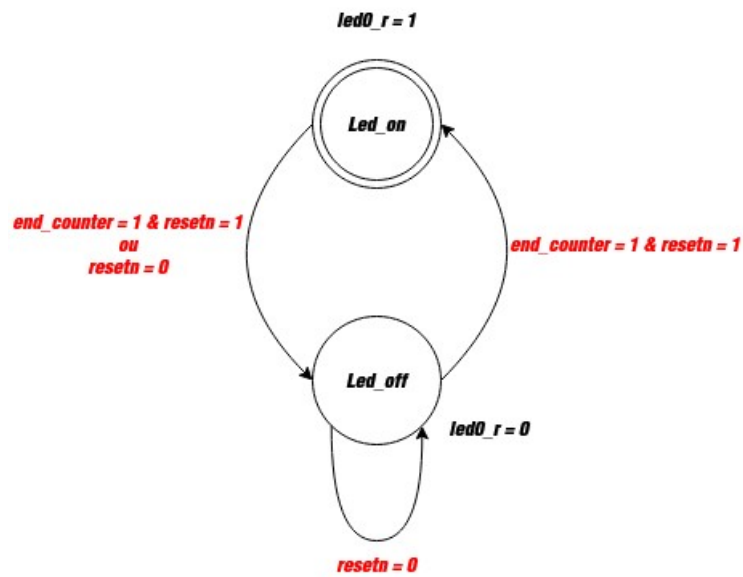
Pour faire clignoter la led, nous avons besoin d'une machine à état avec deux états ; dans l'état 0, la led est éteinte, dans l'état 1, elle est allumée.

Cette machine à état est pilotée par *end_counter* ;

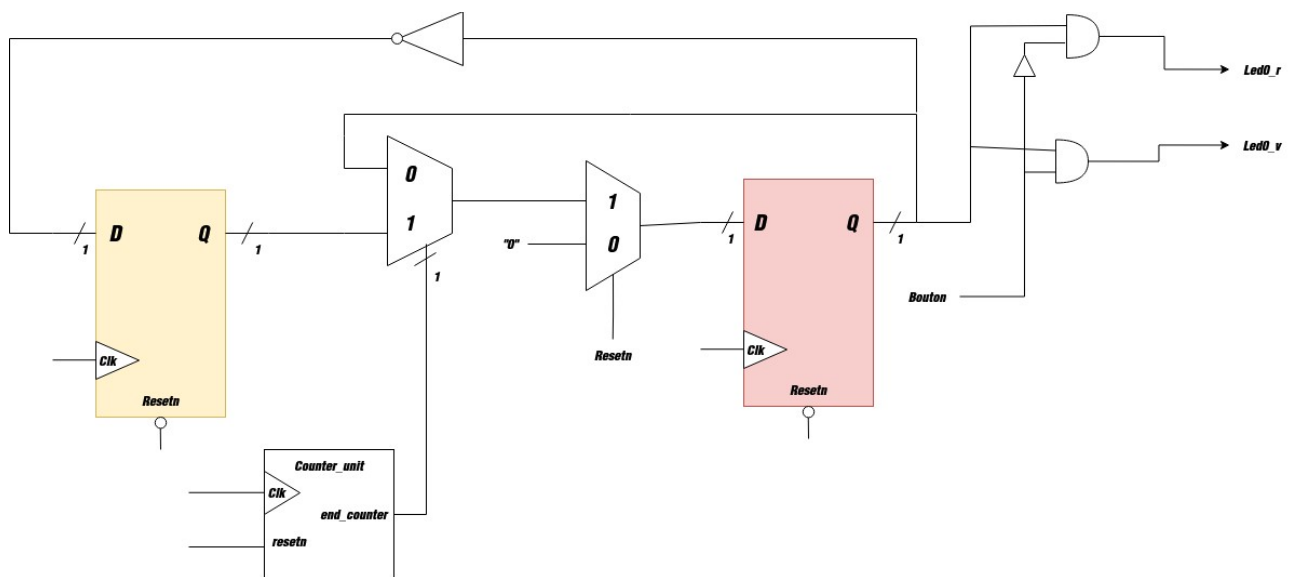
lorsque ce signal est à l'état bas la machine à état maintien son état, s'il vaut 1, elle prend l'état suivant.

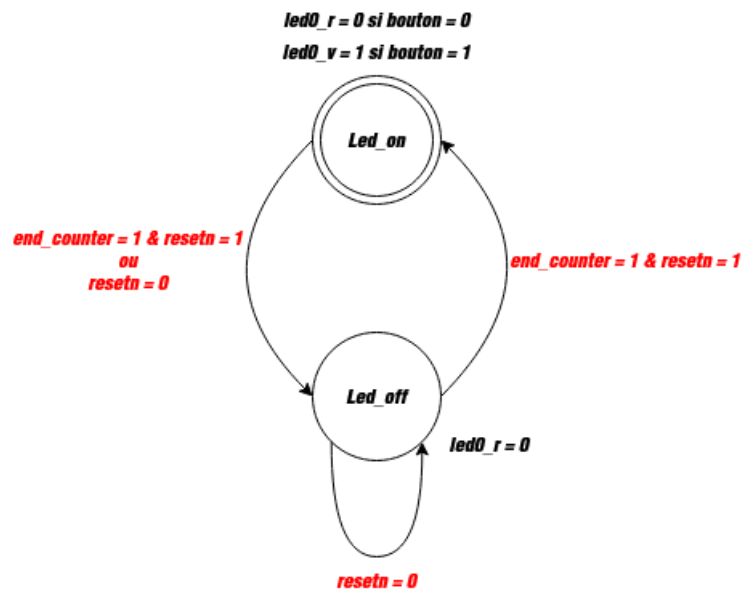
La machine à état est composée de deux bascules, la première mémorise d'état actuel (bascule rouge), la second l'état suivant (bascule jaune).

Les équations logiques pour passer d'un état à l'autre sont une simple inversion car nous n'avons que deux états que l'on peut coder sur un bit.



2. Modifiez votre architecture pour piloter une LED rouge et une LED verte. Lorsque le bouton_0 est appuyé, la LED verte est allumée, sinon la LED rouge est allumée.





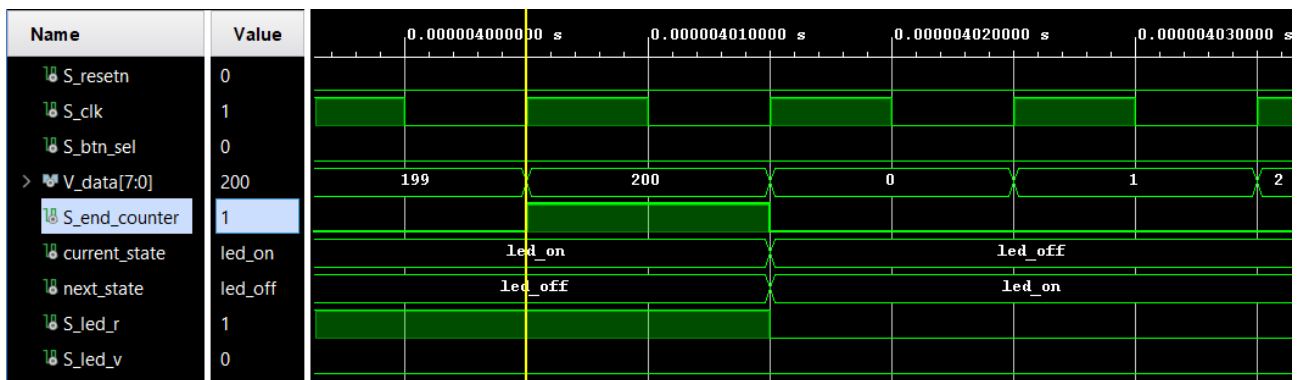
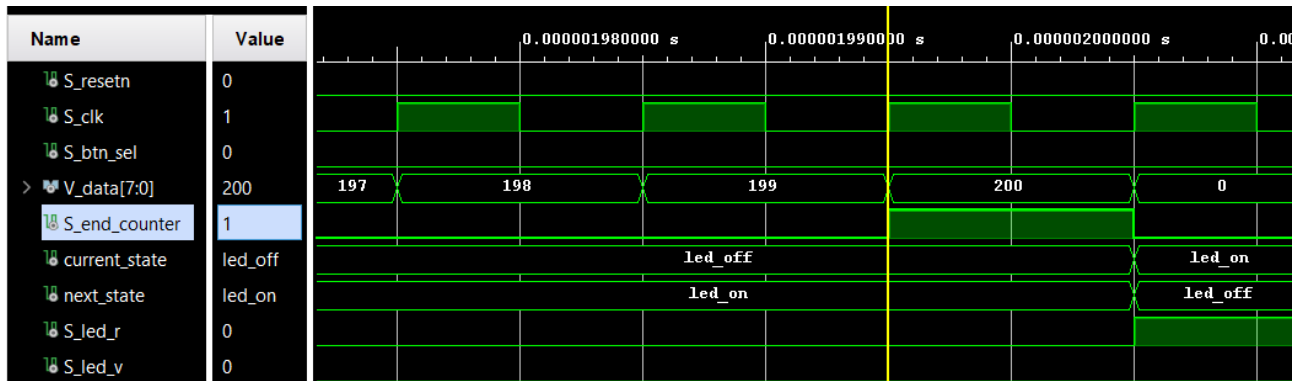
3. Rédigez le code VHDL de votre architecture.

4. Réalisez une simulation en rédigeant un testbench. Que se passe-t-il si le bouton est pressé pendant plus d'un cycle d'horloge ?

Si le bouton est pressé pendant plus d'un cycle d'horloge, la led verte continue de clignoter.

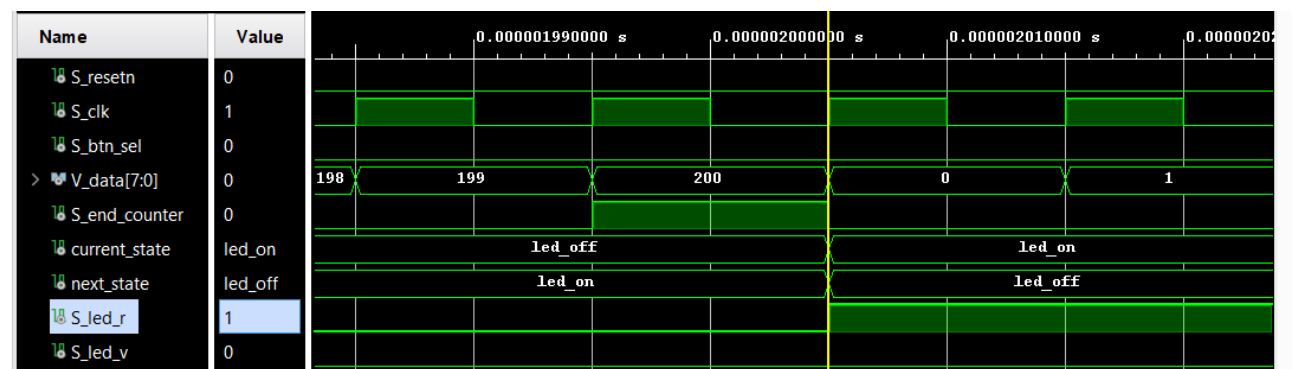
Élément testé	Critère de succès	Ok/Nok
Succession des états	Led_off => led_on et led_on => led_off sur chaque end_compteur = 1.	OK
Extinction des leds	Led_V et Led_R = 0 si current_state = led_off	OK
Allumage led_V	Led_V = 1 si bouton = 1 et current_state = led_on	OK
Allumage led_R	Led_R = 1 si bouton = 0 et current_state = led_on	OK
Resetn	Current_state = led_off et counter_unit réinitialisé.	OK

A/ Validation Succession des états et Extinction des leds.



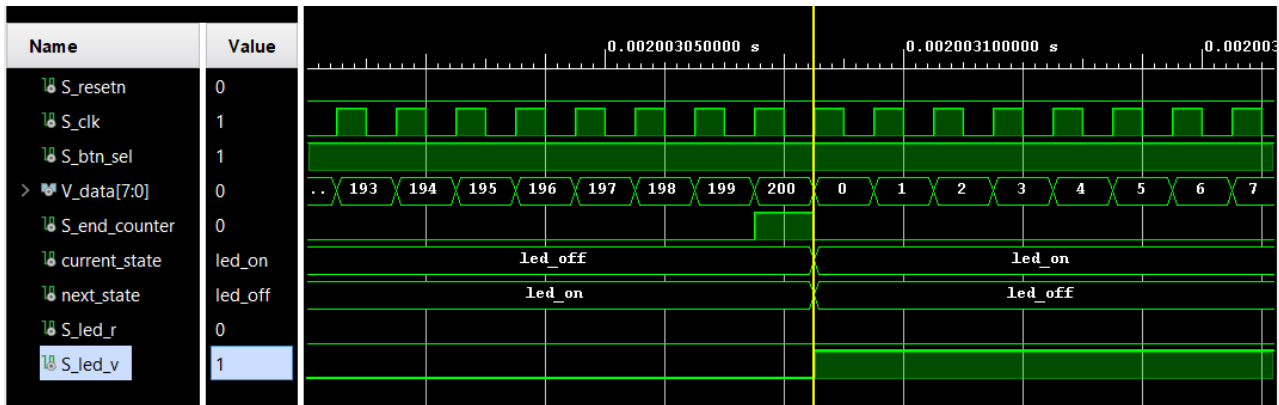
Lorsque S_end_counter vaut 1, l'état actuel est modifié en prenant la valeur de *next_state*

b/ Allumage led_R

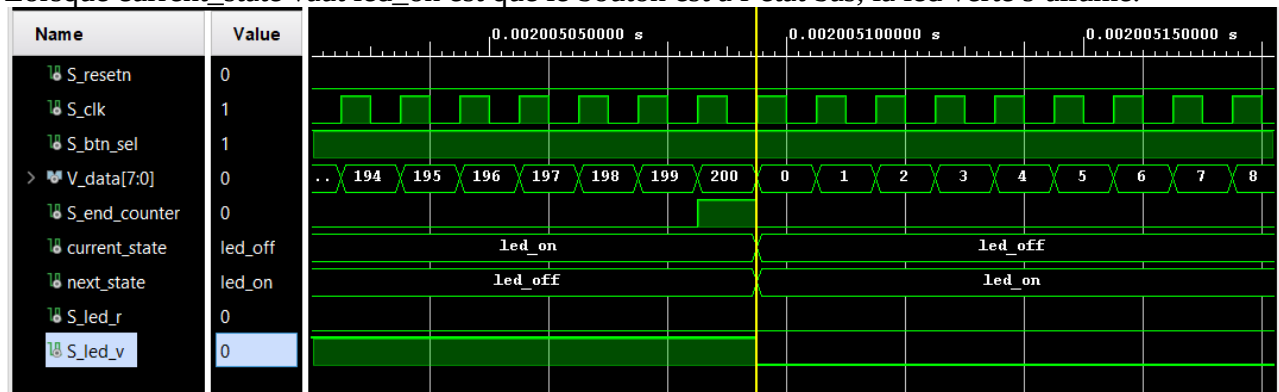


Lorsque current_state vaut led_on est que le bouton est à l'état bas, la led rouge s'allume.

c/ Allumage led_V

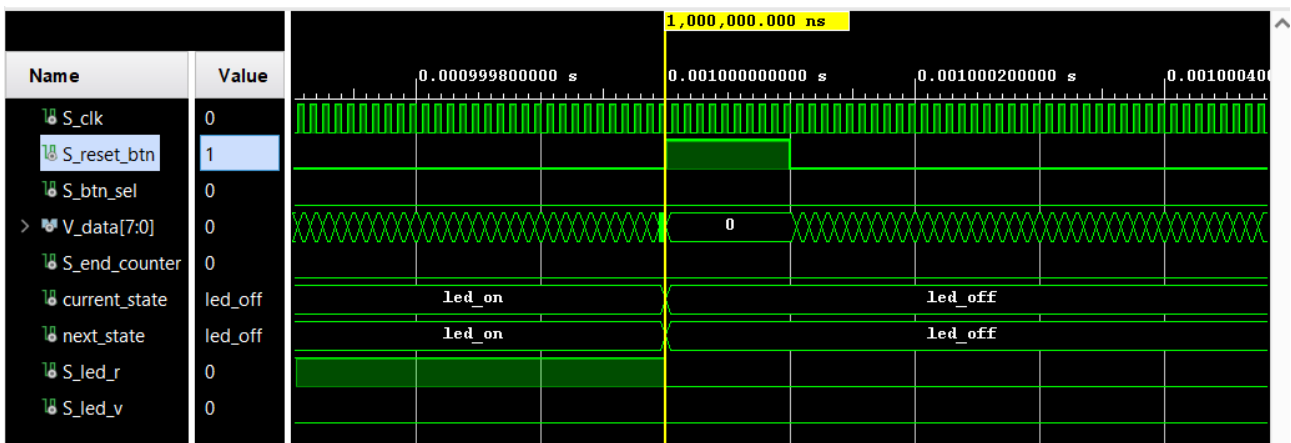


Lorsque current_state vaut led_on est que le bouton est à l'état bas, la led verte s'allume.



sur current_state = led_off, même si le bouton est pressé, les diodes sont éteintes.

d/ Resetrn

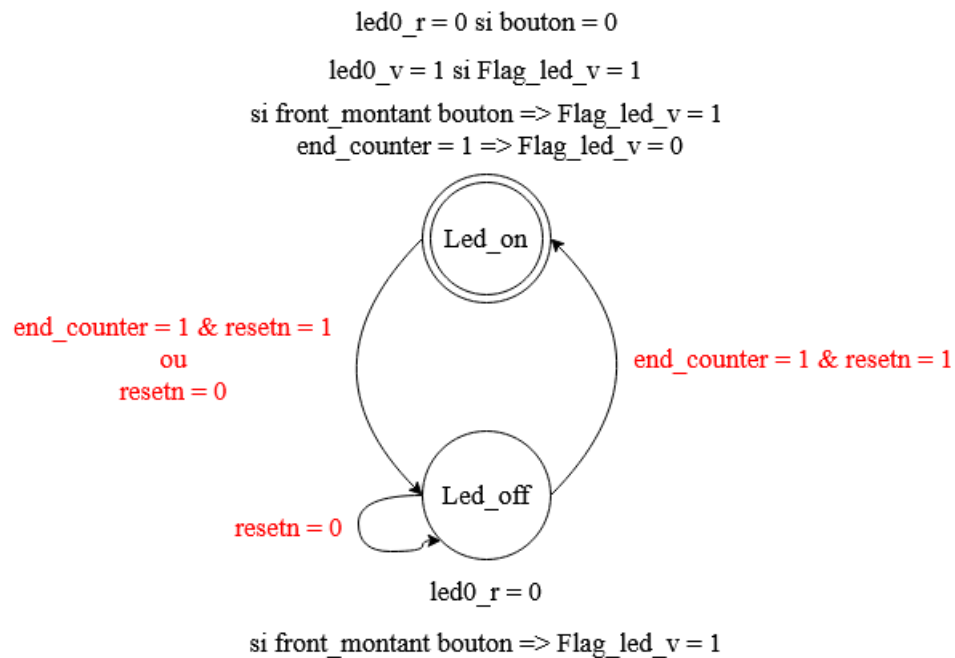


Les Bonnes pratiques en VHDL :

Il faut **toujours** mettre les else des if pour éviter les latches (les état indéterminés).

Il faut **toujours** conserver l'actualisation des états d'une machine à état sur front montant de l'horloge **inconditionnellement**

5. Que faudrait-il faire pour que la LED ne clignote en vert qu'une seule fois même si le bouton est maintenu ?



L'idée est d'utiliser un flag conditionnant l'allumage de la led verte. Si détection d'un front montant du bouton, le flag prend comme valeur 1. Il est passé à 0 lorsque l'on passe dans l'état led_on, après avoir cligné.

Il est aussi possible d'utiliser le signal de pilotage de la led pour mettre à l'état bas le flag de commande de la led verte. C'est la solution que j'avais initialement privilégié mais il est plus logique de ne pas se servir d'un signal de sortie comme d'une entrée. Le flag devra donc être abaissé dans led_on modifiant la couleur de la led immédiatement après pression du bouton, la led verte ne reste pas allumée pendant une durée de clignotement complète.

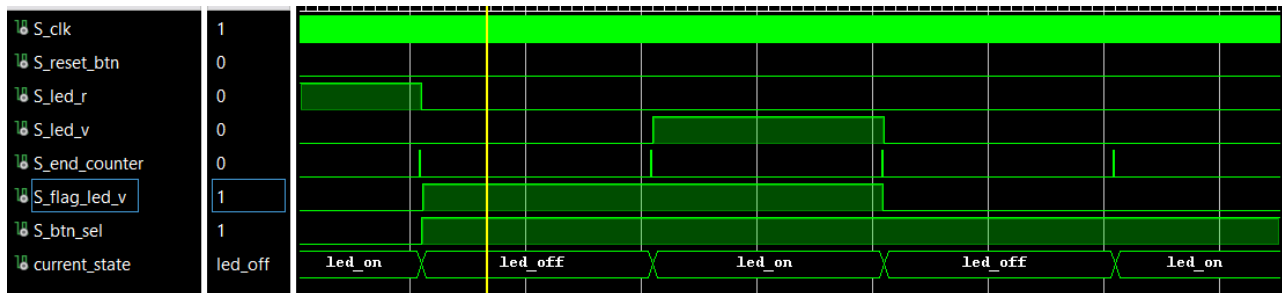
Il n'est pas possible de conditionner la levée du flag avec le passage d'un état à un autre car la variable de détection du front montant n'est à l'état haut que pendant une période d'horloge.

6. Ajoutez cette solution à votre architecture RTL.

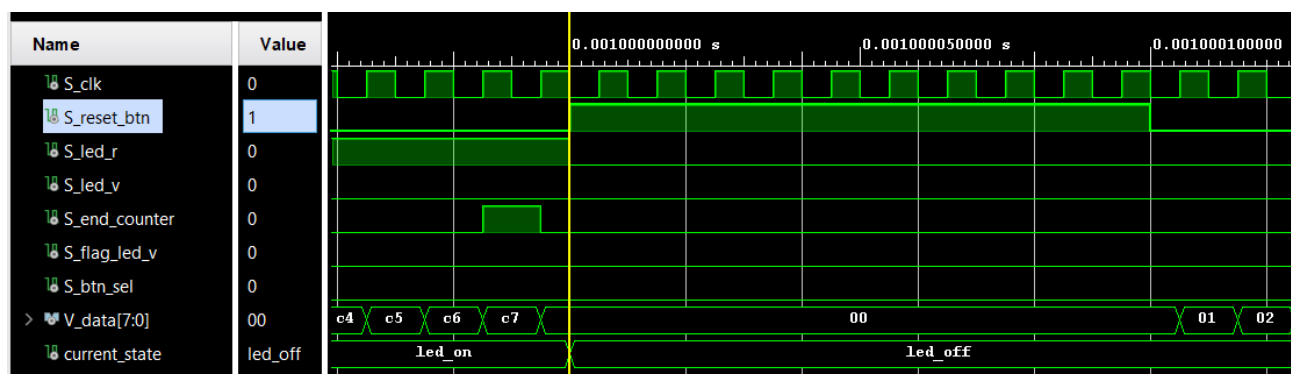
7. Mettez à jour votre code VHDL et vérifiez votre résultat à la simulation.

Élément testé	Critère de succès	Ok/Nok
Succession des états	Led_off => led_on et led_on => led_off sur chaque end_compteur = 1.	OK
Extinction des leds	Led_V et Led_R = 0 si current state = led_off	OK

Allumage led_V	Led_V = 1 si S_flag_led_v = 1 et current_state = led_on	OK
Allumage led_R	Led_R = 1 si bouton = 0 et current_state = led_on	OK
Resetn	Current_state = led_off	OK



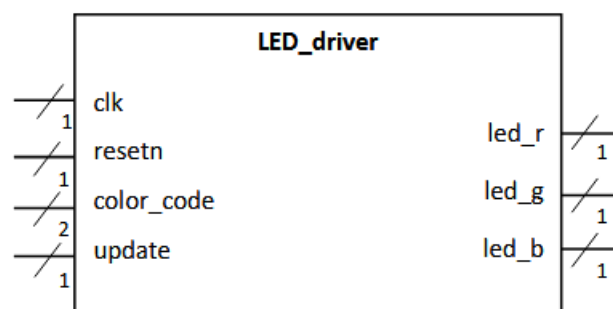
Sur S_btn_sel = '1', et S_end_counter = 1 ; S_flag_led_v passe à l'état haut, il passe à l'état bas une fois que la led verte à clignoté puis aucune diode ne s'allume.



Sur pression du bouton reset, les diodes d'éteignent, le compteur de temporisation passe à 0, et current_state à « led_off ».

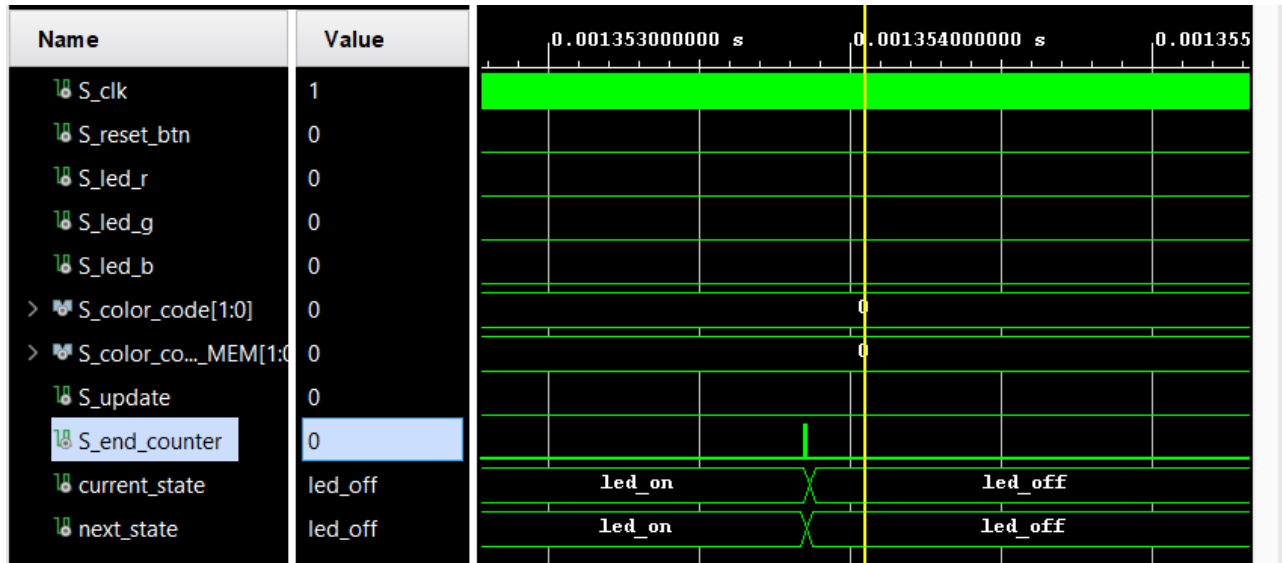
Démonstration Cf : Vidéo jointe.

8. Créez un module de pilotage d'une LED RGB en RTL. Ce dernier doit permettre de faire clignoter une LED RGB connectée en sortie d'une couleur définie par un code couleur donné en entrée. Le changement de couleur de la LED RGB n'a lieu que si un signal update est reçu.

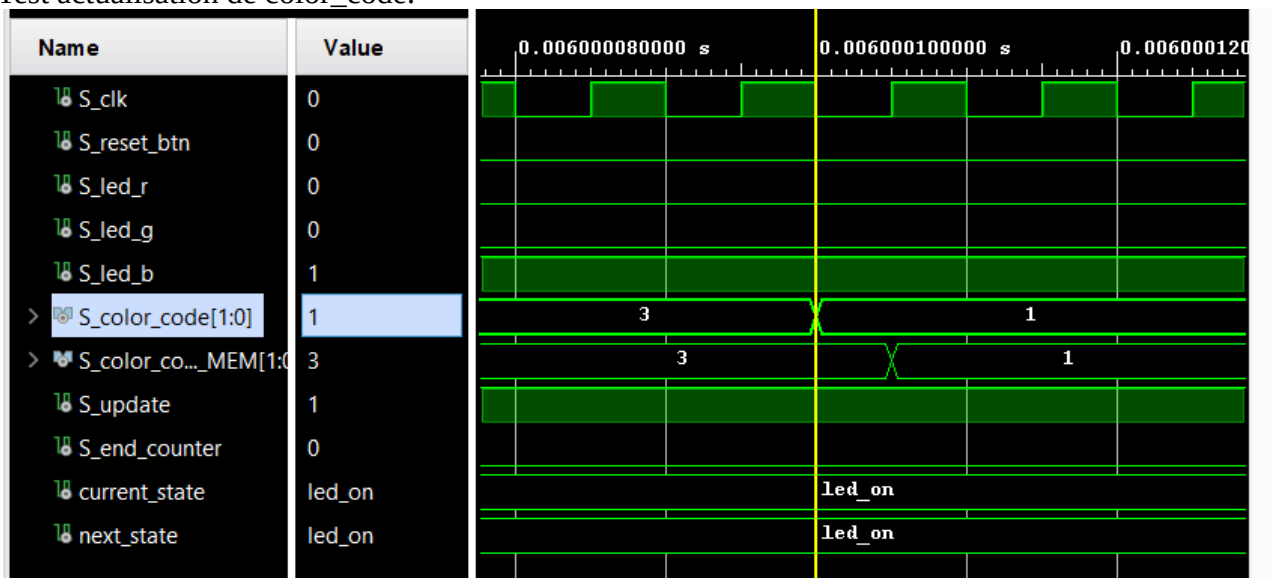


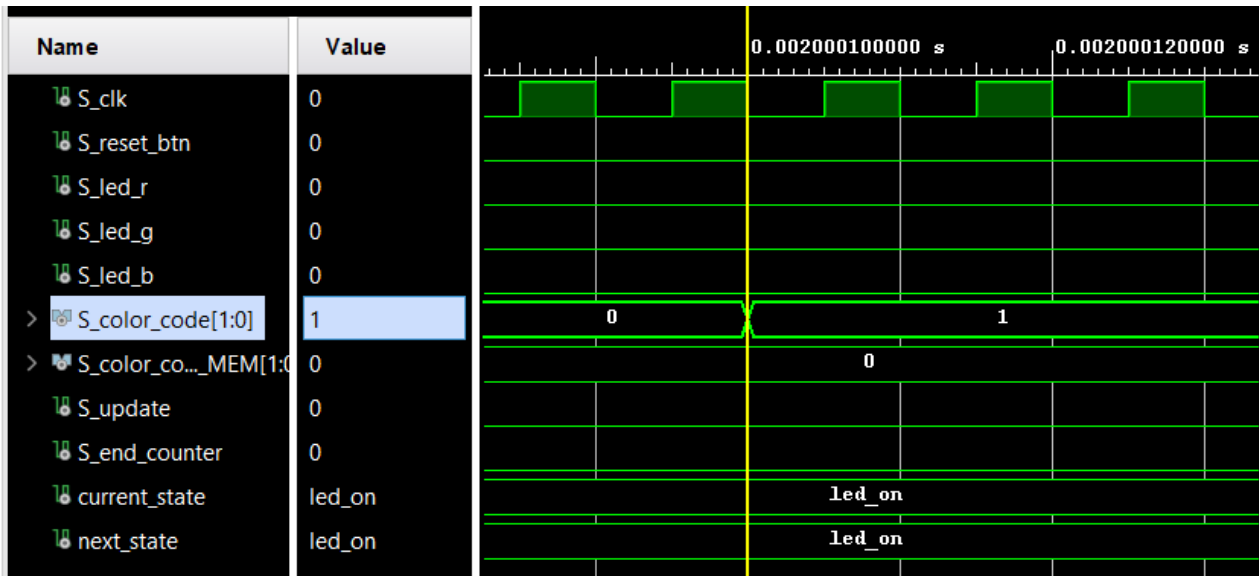
		Bleu est allumée.	
Actualisation color_code.	de	Si update = 1 alors sur rising_edge de l'horloge, on actualise color_code_mem avec la nouvelle valeur.	OK
Resetn		Current_state = led_off color_code_mem = 0	OK

Test de succession des états et extinction des leds :



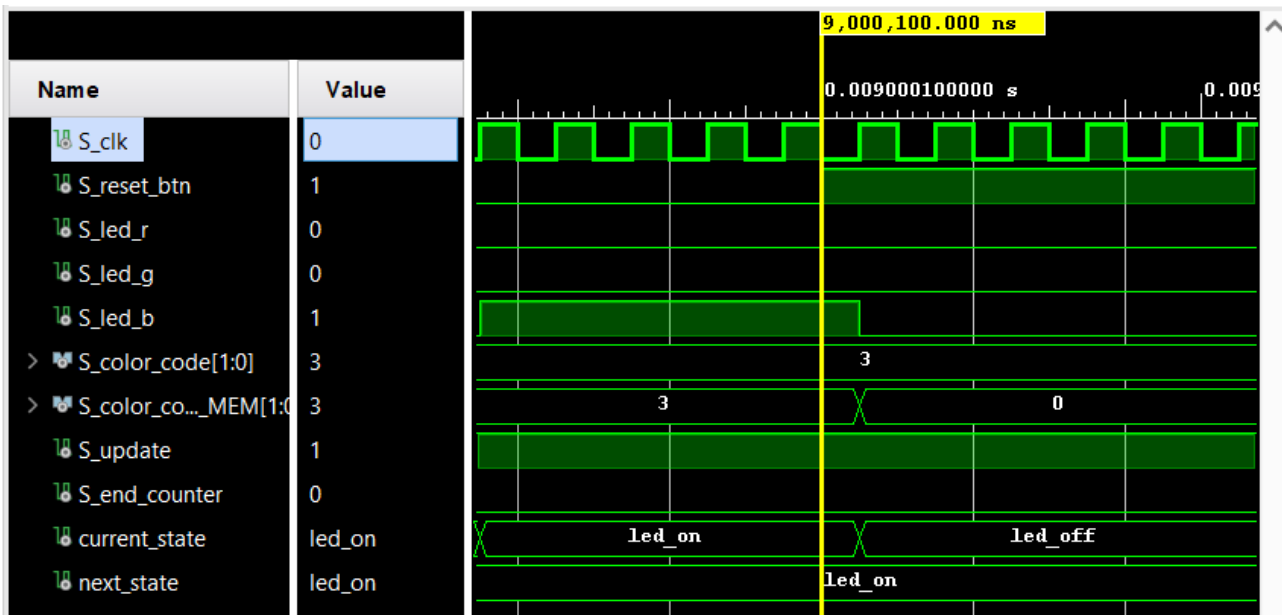
Test actualisation de color_code.





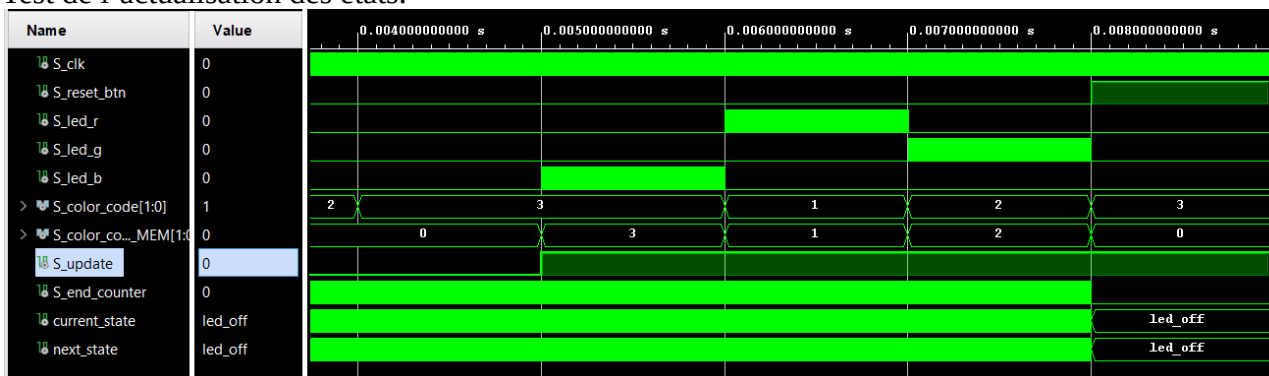
S_color_mem est bien actualisé sur front montant d'horloge, seulement lorsque S_update est actif.

Test de resetn



Sur front montant du reset, on a bien led_off et S_color_mem = 0.

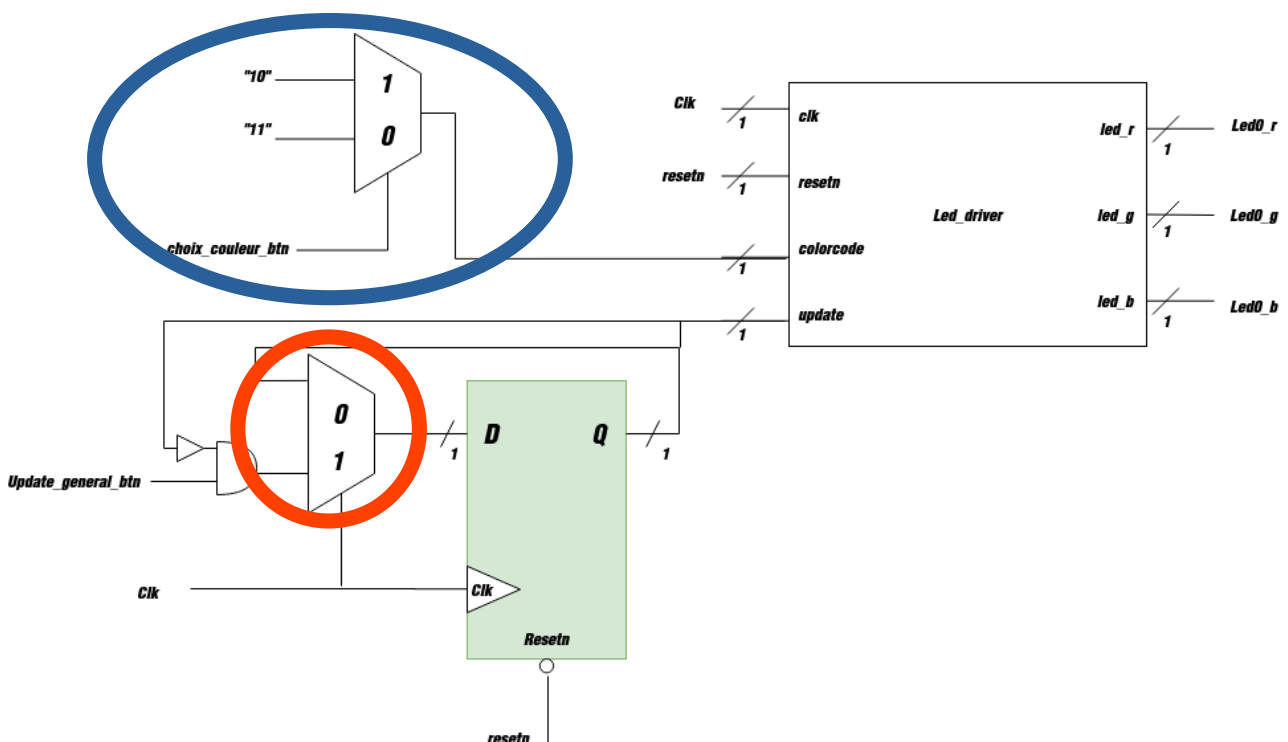
Test de l'actualisation des états.



On constate que lorsque S_update est à l'état haut, S_led_r, S_led_b, et S_led_g s'allument à tout de rôle. Lorsque S_color change.

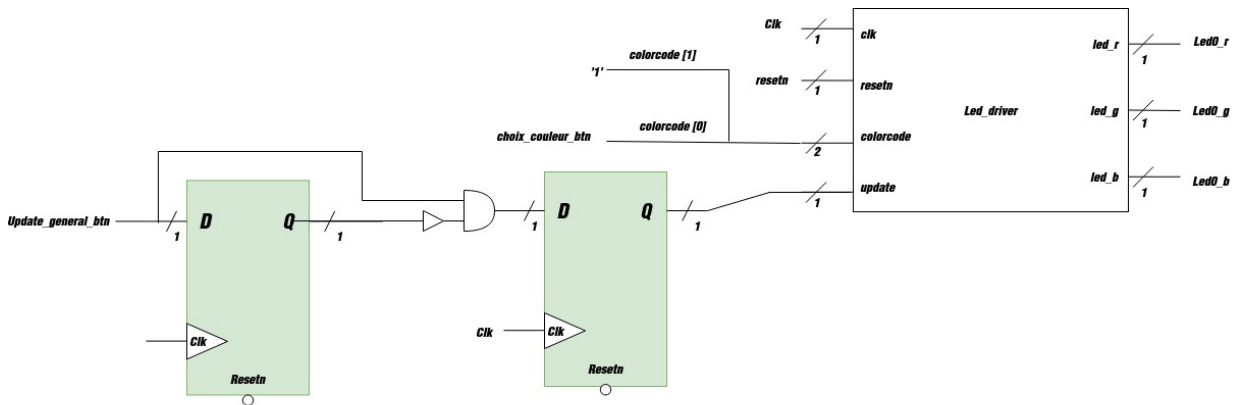
9. Ajouter la logique nécessaire pour piloter les entrées/sorties de votre module.

- Le signal update doit recevoir 1 uniquement lorsque le bouton _0 vient d'être appuyé, maintenir le bouton enfoncé ne doit pas maintenir le signal update à 1
- Le signal color_code doit recevoir soit le code couleur « vert » si le bouton_1 est pressé, il doit recevoir le code couleur « bleu » sinon
- Les LED R, G et B sont connectées avec les couleurs respectives de la LED_0



Le schéma généré par vivado, n'est pas identique car on ne peut pas utiliser le signal d'horloge comme entrée d'un multiplexeur.

Les seules entrées **admissibles** pour le signal d'horloge sont **les entrées horloges des bascules**.



Il est possible d'inverser les deux bascules afin d'avoir une entrée synchrone sur le bouton de mis à jour.

En ce qui concerne le bouton de choix de la couleur, il est simplifié par vivado par rapport au schéma car le MSB de color code ne change pas, il est toujours à 1. Le Lsb de colorcode est la lecture directe du bouton de choix de couleur.

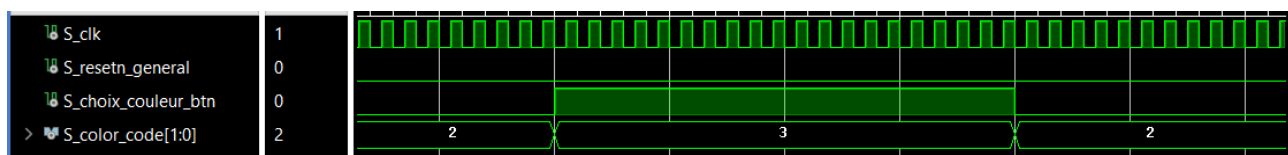
10. Ecrivez le code VHDL correspondant à votre architecture.

11. Ecrivez un testbench pour tester votre design.

12. Vérifier votre résultat à la simulation.

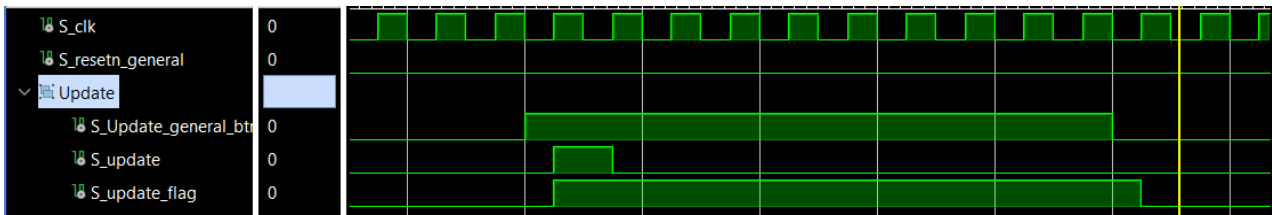
Élément testé	Critère de succès	Ok/Nok
Comportement du bouton choix_couleur_btn	Colorcode = 2 quand choix_couleur_btn = '0' Colorcode = 3 quand choix_couleur_btn = '1'	OK
Comportement de Update_general_btn	La pression de Update_general génère une impulsion d'une période d'horloge sur l'entrée Update de Led_driver.	OK
Reset_general	Update vaut '0'	OK

A/ Vérification du bouton de choix de la couleur :



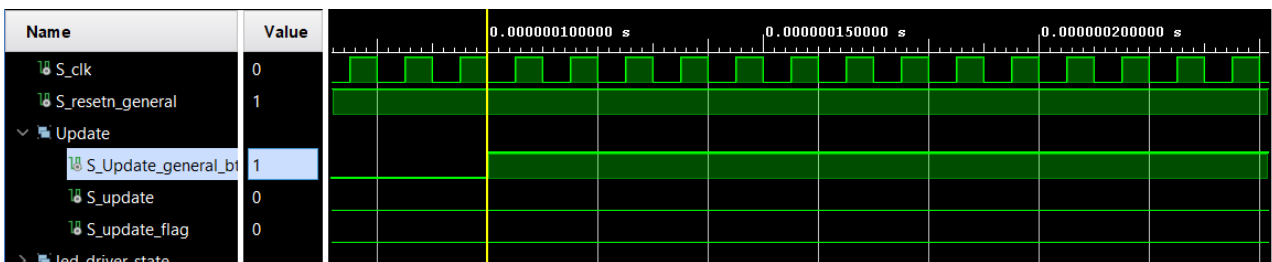
L'actualisation de S_color_code est asynchrone.

B/ Vérification du bouton de choix d'Update.



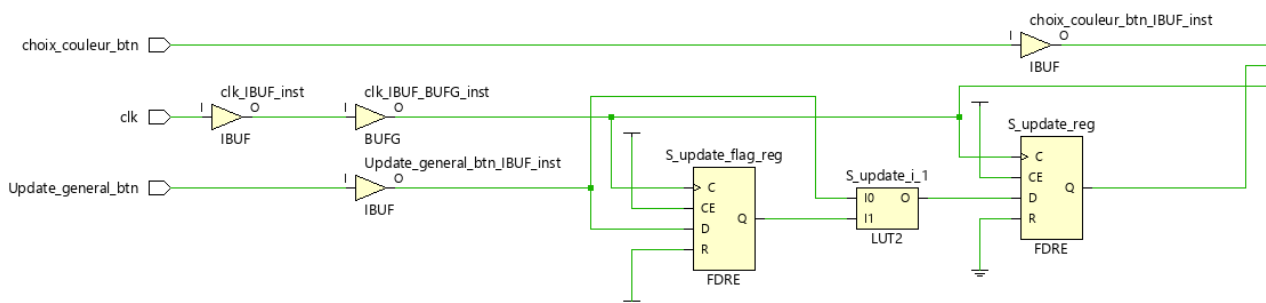
La pression du bouton d'update (S_update_general_flag) génère un pulse d'une durée d'une fréquence d'horloge. Ce pulse n'est pas répété même si le bouton est maintenu.

C/ Vérification du reset



Si reset actif, pas de pulse n'est généré.

13. Réalisez une synthèse et étudiez le rapport de synthèse, les ressources utilisées doivent correspondre à votre schéma RTL.



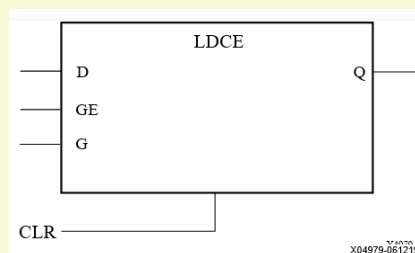
<pre>Report Cell Usage: +-----+-----+ Cell Count +-----+-----+ 1 BUFG 1 2 LUT2 8 3 LUT3 4 4 LUT6 9 5 FDRE 14 6 LDC 2 7 LDCP 1 8 IBUF 3 9 OBUF 3 +-----+-----+</pre> Rapport de synthèse avec un délais de temporisation de 16 us (2000 coups d'horloge)	<pre>Report Cell Usage: +-----+-----+ Cell Count +-----+-----+ 1 BUFG 1 2 CARRY4 7 3 LUT2 13 4 LUT3 4 5 LUT4 2 6 LUT6 32 7 FDRE 34 8 LDC 2 9 LDCP 1 10 IBUF 3 11 OBUF 3 +-----+-----+</pre> Rapport de synthèse avec un délais de temporisation de 2s (250 000 000 coups d'horloge)
--	--

Cf schématique en Pièce-jointe :

- 2 registres et une lut pour le détecteur de front montant sur update. (bleu)
- 2 registres pour la machine à état. (Vert)
- 2 registres et une lut pour mémoriser le choix de la couleur (Rouge)
- 3 registres pour la sortie des leds (Cyan) (Bascule LDCE)

L'entrée des Luts gérant l'état des bascules des leds dépendent de current_state et de S_color_code

- Il y a 28 registres permettant de compter jusqu'à 265 435 456



LDCE : Transparent Latch with Clock Enable and Asynchronous Clear, la bascule recopie sur Q les données entrantes sur D, sur front montant de G (gate) seulement lorsque GE (Gate enable) est actif. Clk reset la bascule.

Dans notre cas, la GE est toujours active, le signal de mise à jour de ces bascules sont issus du vecteur S_color_mem.

14. Effectuez le placement routage et étudiez les rapports.

Setup	Hold
Worst Negative Slack (WNS): 7,809 ns	Worst Hold Slack (WHS): 0,177 ns
Total Negative Slack (TNS): 0,000 ns	Total Hold Slack (THS): 0,000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 13	Total Number of Endpoints: 13

All user specified timing constraints are met.

Timing constraint to count until 200 clock ticks.

Setup	Hold
Worst Negative Slack (WNS): 4,984 ns	Worst Hold Slack (WHS): 0,171 ns
Total Negative Slack (TNS): 0,000 ns	Total Hold Slack (THS): 0,000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 33	Total Number of Endpoints: 33

All user specified timing constraints are met.

Timing constraint to count until 200 000 000 clocks ticks.

Il semble logique que lorsque le nombre d'éléments augmente le temps de propagation augmente (temps effectif). Le slack diminue car le temps effectif augmente alors que le temps voulu reste le même.

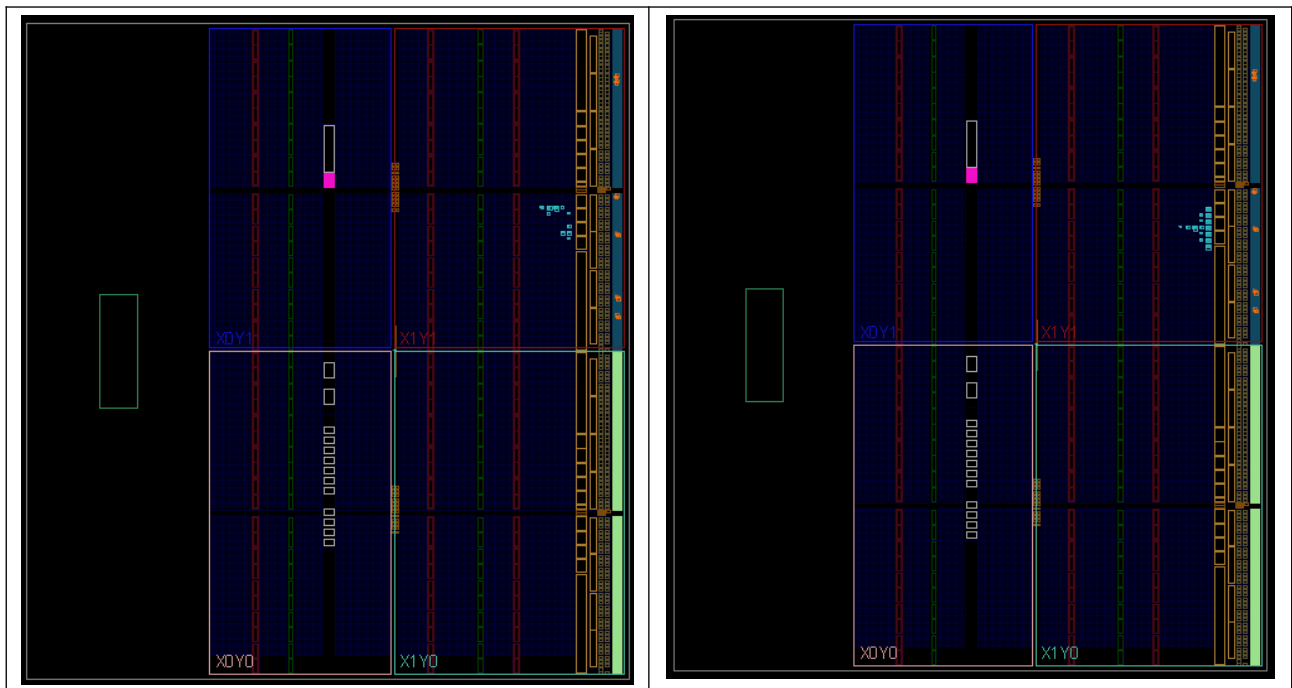
Slack => temps voulu – temps effectif.

Le Slack reste positif, le temps de mise à jour des signaux logiques dans les portes logiques reste inférieur à la période de l'horloge.

```

Max Delay Paths
-----
Slack (MET) :          3.189ns  (required time - arrival time)
Source:        Led_Driver_1/counter_unit_1/V_data_reg[2]/C
                (rising edge-triggered cell FDRE clocked by sys_clk_pin  {rise@0.000ns fall@4.000ns period=8.000ns})
Destination:   Led_Driver_1/counter_unit_1/V_data_reg[25]/D
                (rising edge-triggered cell FDRE clocked by sys_clk_pin  {rise@0.000ns fall@4.000ns period=8.000ns})
Path Group:    sys_clk_pin
Path Type:     Setup (Max at Slow Process Corner)
Requirement:   8.000ns  (sys_clk_pin rise@8.000ns - sys_clk_pin rise@0.000ns)
Data Path Delay: 4.854ns  (logic 2.472ns (50.928%) route 2.382ns (49.072%))
Logic Levels:  11  (CARRY4=7 LUT6=4)
Clock Path Skew: -0.031ns  (DCD - SCD + CPR)
  Destination Clock Delay (DCD):  4.910ns = ( 12.910 - 8.000 )
  Source Clock Delay (SCD):        5.369ns
  Clock Pessimism Removal (CPR):    0.429ns
Clock Uncertainty: 0.035ns  ((TSJ^2 + TIJ^2)^1/2 + DJ) / 2 + PE
  Total System Jitter (TSJ):        0.071ns
  Total Input Jitter (TIJ):          0.000ns
  Discrete Jitter (DJ):              0.000ns
  Phase Error (PE):                 0.000ns
  
```

Le rapport timing indique que le chemin critique dure 4,854 ns. (3,2ns = 8ns – 4,8ns)

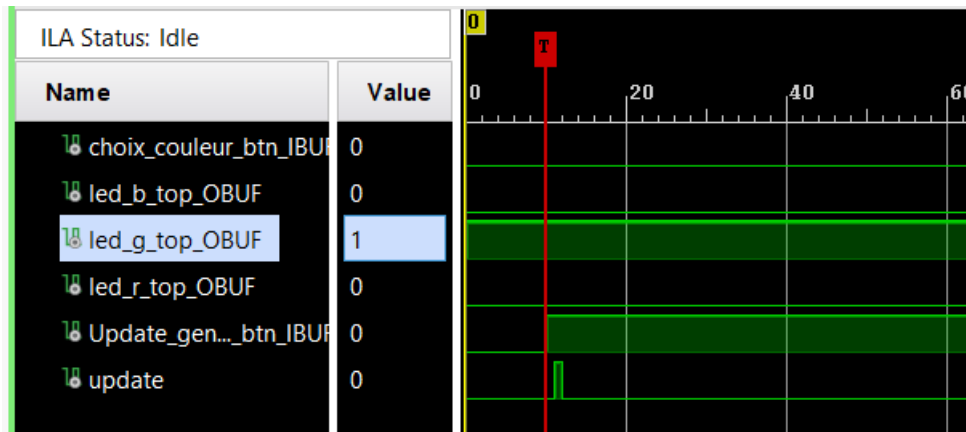


Design avec 200 Bascules.

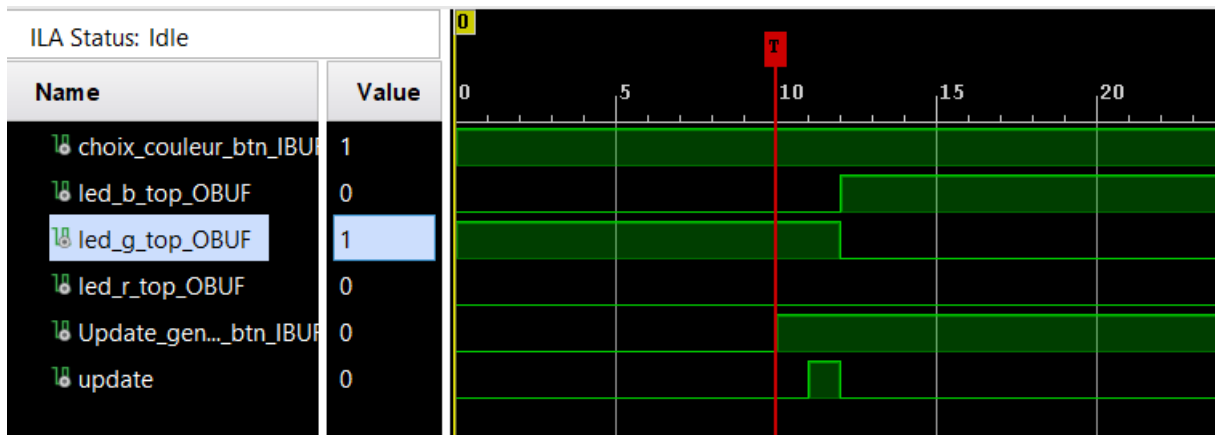
15. Générez le bitstream et vérifiez que vous avez le comportement attendu sur carte.

J'obtiens bien le comportement attendu sur carte, sur pression sur bouton 0 (update) la couleur de la led clignotant toutes les deux secondes est verte. Lorsque le bouton 1 (choix couleur) est appuyé en même temps que le bouton 0, la led devient bleue. La diode rouge ne clignote jamais. CF Vidéo de démonstration.

16. Utilisation de sondes ILA pour vérifier le bon comportement sur la carte.



Maintien du choix d'allumage de la led verte lorsque le bouton Update est pressé et que le bouton de choix de la couleur est inactif. Le signal envoyé à l'unité de contrôle des leds est bien une impulsion d'une durée d'une période d'horloge.

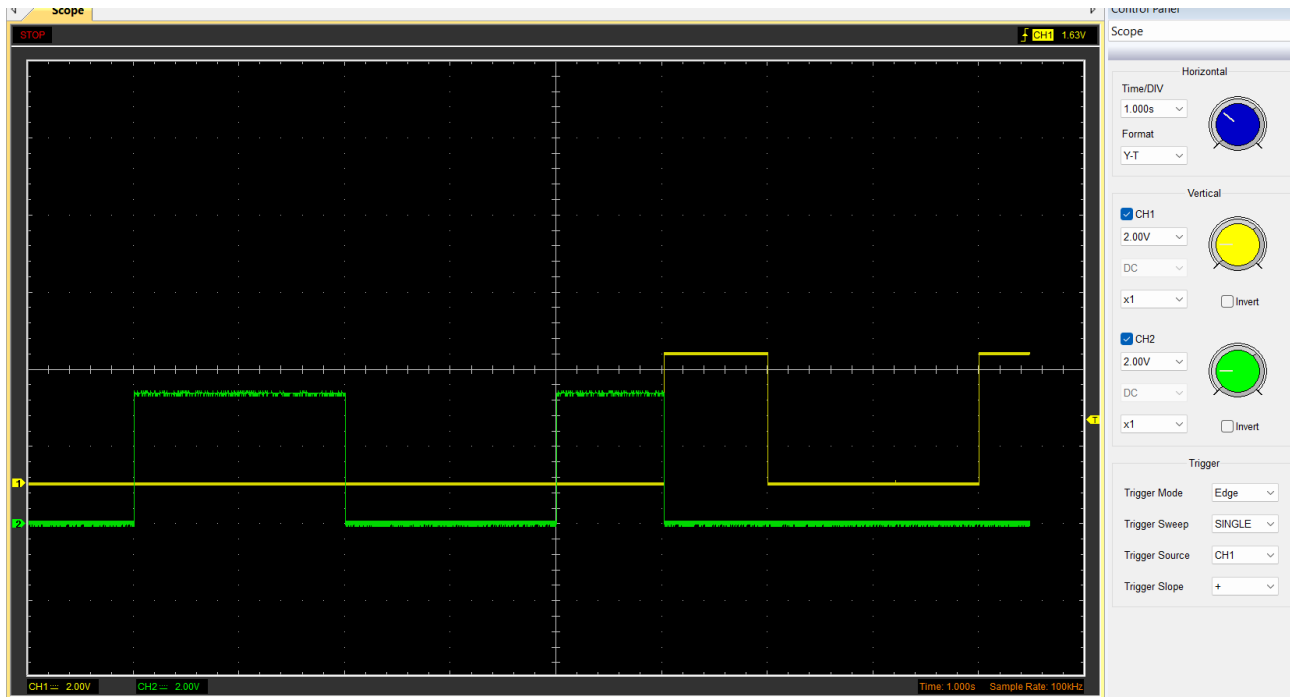


Lorsque le bouton de sélection de la couleur et de l'actualisation sont pressés, le couleur de la diode change du vert au bleu.

16. Vérification à l'oscilloscope

La vérification de la génération de l'impulsion d'update à l'oscilloscope est impossible car la fréquence maximale d'échantillonnage est 1Mhz sur le calibre 10ns.

Vérification des durées d'allumage des leds à l'oscilloscope.



Mis à jours de la couleur demandé pendant allumage des leds, le channel 1 est la led verte, le channel 2, la led bleue. La durée d'impulsion est conforme à la durée demandé : 2 sec.